

AI

AI use and design judgment

AI tools supported architecture brainstorming, implementation, debugging, test verification, and documentation. The submitted design remains the student's responsibility; the decisions below are stated in terms that can be explained from the source code.

AI tools used

Tool	How it was used
Claude by Anthropic	Proposed the initial package split, generated the first implementation and documentation drafts, exercised example scenarios, and helped diagnose a Java classpath issue in PowerShell.
ChatGPT Work with Codex by OpenAI	Reviewed the assignment and source, recompiled the project, ran a deterministic 12-test suite plus a Driver scenario, and rewrote the Design, Tests, and AI documents for clarity and layout.

Three important AI interactions

1 Architecture and implementation

The assignment was provided to Claude, which proposed a reusable assignment2.core package and a Mastermind-specific assignment2.mastermind package. The proposal separated secret generation, validation, evaluation, feedback, history, and user interaction behind small contracts, then generated the initial Java implementation. The final source retains this split because each interface maps to a distinct responsibility used by GuessingGameEngine.

2 Build and classpath debugging

When java assignment2.Driver mastermind failed from a VS Code PowerShell terminal, Claude identified two causes: the Bash-only \$(find ...) build command did not work in PowerShell, and the compiled output directory was not on the Java classpath. The corrected approach compiles into an output folder and either runs with -cp out or runs from that folder.

3 Verification and document redesign

ChatGPT with Codex checked the final source against the assignment, compiled it through the JDK compiler module, and ran 12 deterministic tests. It then replaced the original dense documents with a routed architecture diagram, responsibility matrices, requirement coverage, concise test tables, and this AI-use summary.

AI suggestion reviewed and rejected

Suggestion

Claude AI suggested creating a separate Command interface and individual command classes for HISTORY and REVEAL instead of handling those commands directly in GuessingGameEngine.

Decision

I rejected this suggestion for Phase I because the program currently has only two simple commands, and the existing code handles them clearly with two if statements. Two commands are not complicated enough to justify several additional classes. Adding a Command interface and several new classes would make the design harder to follow without providing much benefit. If Phase II introduces many commands or allows different games to define their own commands, separate command classes may become useful and I would reconsider the approach.

Criterion	Evaluation
Current complexity	Only HISTORY and test-only REVEAL are recognized.
Benefit of abstraction now	Small; it would move two direct comparisons into several new types.
Cost	More indirection, construction logic, and explanation during checkout.
Revisit when	Commands vary by game, become numerous, or need dynamic registration.

Responsibility for the submission

AI output was treated as a draft to inspect rather than as an authority. The final architecture can be defended from the code: the engine depends on interfaces, Mastermind owns its scoring and feedback rules, invalid guesses do not consume attempts, and deterministic secret generation makes tests repeatable. The student remains responsible for explaining, critiquing, and extending every submitted line.

AI use and decisions

Assignment 2, Part II | Transparent record of assistance and architectural judgment.

Assistance used

Stage	AI contribution	Reviewable result
Phase I (previous submission)	Claude assisted with the initial core/Mastermind implementation; ChatGPT Work with Codex reviewed and rewrote the Phase I reports.	The original source and Design.pdf remain available in this project.
Part II extension	ChatGPT Work with Codex read the handout and Phase I source, proposed the design, wrote the Wordle classes and shared replay refactoring, and drafted the documentation and PDFs.	Java files, word list, architecture report, and test harness are included.
Verification	ChatGPT Work with Codex compiled with OpenJDK 17 and ran 20 checks plus Driver sessions.	All automated checks passed; cases and commands appear in TESTS.pdf.

Suggestions accepted for this draft

The AI suggested keeping the Phase I round engine and injecting a Wordle-specific validator, evaluator, feedback type, and whole-word generator. The resulting code does this. It also suggested moving replay from MastermindGame into GameSession so both games use one coordinator. This decision is visible in both game classes and GameSession.

Independent rules that guided implementation

The handout requires a provided external word list, valid dictionary guesses, six valid attempts, positional C/P/A feedback and no double counting. The implementation separates dictionary and game rules so those requirements can be checked directly. The AI identified an inconsistency in the handout APPLE/ALLEY example; the duplicate-occurrence and position rules were applied consistently and are covered by test W02.

Judgment and limitations

Suggestion not adopted

The prior AI suggestion to create a Command interface and separate classes for HISTORY and REVEAL remains unadopted. The shared engine still handles these two commands with direct checks. Wordle adds no game-specific command, so another command hierarchy would add classes without clarifying the current flow.

Evaluation	Reason
Current benefit	Small: both commands have identical nonattempt behavior in both games.
Current cost	More classes, wiring, and indirection for two checks.
Revisit if	Games gain different commands or commands must be registered dynamically.

Tradeoffs made in the implementation

The 534-word file is a curated, external list rather than a comprehensive dictionary. The same list supplies allowed guesses and possible secrets. That is a simple, auditable interpretation of the assignment. A larger provided dictionary can replace the file without changing source. The engine adds a word/code label for clear output while retaining the old Mastermind constructor.